

Nama : Khania Puji Auliya
NIM : 254107020236
Kelas : 1G
Prodi : D-IV Teknik Informatika

LAPORAN JOBSHEET 14

PERCOBAAN 1

➤ Kode Program MahasiswaT15

```
public class MahasiswaTr15 {  
    String nim;  
    String nama;  
    String kelas;  
    double ipk;  
  
    public MahasiswaTr15() {  
    }  
    public MahasiswaTr15(String nim, String nama, String kelas, double ipk) {  
        this.nim = nim;  
        this.nama = nama;  
        this.kelas = kelas;  
        this.ipk = ipk;  
    }  
    public void tampilInformasi() {  
        System.out.println(  
            "NIM: " + this.nim + " | " +  
            "Nama: " + this.nama + " | " +  
            "Kelas: " + this.kelas + " | " +  
            "IPK: " + this.ipk);  
    }  
}
```

➤ Kode Program NodeT15

```
public class NodeT15 {  
    MahasiswaTr15 mahasiswa;  
    NodeT15 left, right;  
  
    public NodeT15() {  
    }  
  
    public NodeT15(MahasiswaTr15 mahasiswa) {  
        this.mahasiswa = mahasiswa;  
        left = right = null;  
    }  
}
```

```

    }
}

```

➤ Kode Program BinaryTree15

```

public class BinaryTree15 {
    NodeT15 root;

    public BinaryTree15() {
        root = null;
    }
    public boolean isEmpty() {
        return root == null;
    }
    public void add(MahasiswaTr15 mahasiswa) {
        NodeT15 newNode = new NodeT15(mahasiswa);
        if (isEmpty()) {
            root = newNode;
        } else {
            NodeT15 current = root;
            NodeT15 parent = null;
            while (true) {
                parent = current;
                if (mahasiswa.ipk < current.mahasiswa.ipk) {
                    current = current.left;
                    if (current == null) {
                        parent.left = newNode;
                        return;
                    }
                } else {
                    current = current.right;
                    if (current == null) {
                        parent.right = newNode;
                        return;
                    }
                }
            }
        }
    }

    public boolean find(double ipk) {
        boolean result = false;
        NodeT15 current = root;
        while (current != null) {
            if (current.mahasiswa.ipk == ipk) {
                result = true;
            }
        }
    }
}

```

```

        break;
    } else if (ipk > current.mahasiswa.ipk) {
        current = current.right;
    } else {
        current = current.left;
    }
}
return result;
}
public void traversePreOrder (NodeT15 node) {
    if (node != null) {
        node.mahasiswa.tampilInformasi();
        traversePreOrder(node.left);
        traversePreOrder(node.right);
    }
}
public void traverseInOrder (NodeT15 node) {
    if (node != null) {
        traverseInOrder(node.left);
        node.mahasiswa.tampilInformasi();
        traverseInOrder(node.right);
    }
}
public void traversePostOrder (NodeT15 node) {
    if (node != null) {
        traversePostOrder(node.left);
        traversePostOrder(node.right);
        node.mahasiswa.tampilInformasi();
    }
}
public NodeT15 getSuccessor (NodeT15 del) {
    NodeT15 successor = del.right;
    NodeT15 successorParent = del;
    while (successor.left != null) {
        successorParent = successor;
        successor = successor.left;
    }
    if (successor != del.right) {
        successorParent.left = successor.right;
        successor.right = del.right;
    }
    return successor;
}
public void delete (double ipk) {

```

```

if (isEmpty()) {
    System.out.println("Binary tree kosong");
    return;
}
NodeT15 parent = root;
NodeT15 current = root;
boolean isLeftChild = false;
while (current != null) {
    if (current.mahasiswa.ipk == ipk) {
        break;
    } else if (ipk < current.mahasiswa.ipk) {
        parent = current;
        current = current.left;
        isLeftChild = true;
    } else if (ipk > current.mahasiswa.ipk) {
        parent = current;
        current = current.right;
        isLeftChild = false;
    }
}
if (current == null) {
    System.out.println("Data tidak ditemukan");
    return;
} else {
    if (current.left == null && current.right == null) {
        if (current == root) {
            root = null;
        } else {
            if (isLeftChild) {
                parent.left = null;
            } else {
                parent.right = null;
            }
        }
    } else if (current.left == null) {
        if (current == root) {
            root = current.right;
        } else {
            if (isLeftChild) {
                parent.left = current.right;
            } else {
                parent.right = current.right;
            }
        }
    }
}

```

$$\}$$

```
public class BinaryTreeMain15 {
    public static void main(String[] args) {
        BinaryTree15 bst = new BinaryTree15();

        bst.add(new MahasiswaTr15("244160121", "Ali", "A", 3.57));
        bst.add(new MahasiswaTr15("244160221", "Badar", "B", 3.85));
        bst.add(new MahasiswaTr15("244160185", "Candra", "C", 3.21));
        bst.add(new MahasiswaTr15("244160220", "Dewi", "B", 3.54));

        System.out.println("\nDaftar semua mahasiswa (in order traversal):");
        bst.traverseInOrder(bst.root);

        System.out.println("\nPencarian data mahasiswa:");
        System.out.print("Cari mahasiswa dengan ipk: 3.54: ");
    }
}
```

```

String hasilCari = bst.find(3.54)? "Ditemukan":"Tidak ditemukan";
System.out.println(hasilCari);

System.out.print("Cari mahasiswa dengan ipk: 3.22: ");
hasilCari = bst.find(3.22)? "Ditemukan":"Tidak ditemukan";
System.out.println(hasilCari);

bst.add(new MahasiswaTr15("244160131", "Devi", "A", 3.72));
bst.add(new MahasiswaTr15("244160205", "Ehsan", "D", 3.37));
bst.add(new MahasiswaTr15("244160170", "Fizi", "B", 3.46));
System.out.println("\nDaftar semua mahasiswa setelah penambahan 3 mahasiswa:");
System.out.println("InOrder Traversal:");
bst.traverseInOrder(bst.root);
System.out.println("\nPreOrder Traversal:");
bst.traversePreOrder(bst.root);
System.out.println("\nPostOrder Traversal:");
bst.traversePostOrder(bst.root);

System.out.println("\nPenghapusan data mahasiswa");
bst.delete(3.57);
System.out.println("\nDaftar semua mahasiswa setelah penghapusan 1 mahasiswa (in
order traversal");
bst.traverseInOrder(bst.root);
}
}

```

➤ Hasil Program

Daftar semua mahasiswa (in order traversal):
 NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21
 NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54
 NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57
 NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

Pencarian data mahasiswa:

Cari mahasiswa dengan ipk: 3.54: Ditemukan

Cari mahasiswa dengan ipk: 3.22: Tidak ditemukan

Daftar semua mahasiswa setelah penambahan 3 mahasiswa:

InOrder Traversal:

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21
 NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37
 NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46
 NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54
 NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72
NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

PreOrder Traversal:

NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57
NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21
NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54
NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37
NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46
NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85
NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

PostOrder Traversal:

NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46
NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37
NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54
NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21
NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72
NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85
NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57

Penghapusan data mahasiswa

Jika 2 anak, current =

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal)

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21
NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37
NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46
NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54
NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72
NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

Pertanyaan Percobaan

1. Mengapa dalam binary search tree proses pencarian data bisa lebih efektif dilakukan dibanding binary tree biasa?
2. Untuk apakah di class Node, kegunaan dari atribut left dan right?
3. a. Untuk apakah kegunaan dari atribut root di dalam class BinaryTree?
b. Ketika objek tree pertama kali dibuat, apakah nilai dari root?
4. Ketika tree masih kosong, dan akan ditambahkan sebuah node baru, proses apa yang akan terjadi?

5. Perhatikan method add(), di dalamnya terdapat baris program seperti di bawah ini.

Jelaskan secara detil untuk apa baris program tersebut?

```
parent = current;
if (mahasiswa.ipk < current.mahasiswa.ipk) {
    current = current.left;
    if (current == null) {
        parent.left = newNode;
        return;
    }
} else {
    current = current.right;
    if (current == null) {
        parent.right = newNode;
        return;
    }
}
```

6. Jelaskan langkah-langkah pada method delete() saat menghapus sebuah node yang memiliki dua anak. Bagaimana method getSuccessor() membantu dalam proses ini?

Jawaban

1. Binary Search Tree lebih efektif karena data tersusun terurut. Pencarian cukup bergerak ke kiri atau kanan sesuai nilai yang dicari sehingga lebih cepat dibanding binary tree biasa.
2. Pada class Node, atribut left dan right digunakan untuk menyimpan referensi atau alamat node anak.
 - left menunjuk ke node anak kiri yang memiliki nilai lebih kecil dari parent.
 - right menunjuk ke node anak kanan yang memiliki nilai lebih besar dari parent.
3. a. Atribut root pada class BinaryTree digunakan sebagai titik awal atau akar dari binary tree. Semua proses seperti penambahan, pencarian, traversal, dan penghapusan node dimulai dari root.
b. Ketika objek tree pertama kali dibuat, nilai root adalah null karena tree masih kosong dan belum memiliki node.
4. Ketika tree masih kosong lalu ditambahkan sebuah node baru, node tersebut akan langsung menjadi root. Hal ini terjadi pada method add() saat kondisi isEmpty() bernilai true, sehingga root = newNode.
5. Baris program tersebut digunakan untuk membandingkan IPK data baru dengan IPK node saat ini.
 - Jika IPK lebih kecil, pencarian posisi dilanjutkan ke subtree kiri (current.left).
 - Jika IPK lebih besar atau sama, pencarian dilanjutkan ke subtree kanan (current.right).

Ketika ditemukan posisi kosong (`current == null`), node baru akan ditempatkan:

- di sebelah kiri parent dengan `parent.left = newNode`
- atau di sebelah kanan parent dengan `parent.right = newNode`

6. Pada method `delete()`, jika node yang dihapus memiliki dua anak, prosesnya dilakukan dengan mencari node pengganti (successor). Langkah-langkahnya:

- Mencari node yang akan dihapus.
- Memanggil method `getSuccessor(current)` untuk mencari successor.
- Successor adalah node dengan nilai terkecil pada subtree kanan dari node yang dihapus.
- Setelah successor ditemukan, posisi node yang dihapus digantikan oleh successor.
- Subtree kiri dari node lama dipindahkan ke successor agar struktur BST tetap valid.

Method `getSuccessor()` membantu proses penghapusan node yang memiliki dua anak dengan mencari node paling kiri pada subtree kanan. Node tersebut dipilih karena memiliki nilai terkecil yang lebih besar dari node yang dihapus jadi struktur tetap terurut.

TAHAPAN PERCOBAAN

➤ Kode Program `BinaryTreeArray15`

```
public class BinaryTreeArray15 {
    MahasiswaTr15[] dataMahasiswaTr15s;
    int idxLast;

    public BinaryTreeArray15() {
        this.dataMahasiswaTr15s = new MahasiswaTr15[10];
    }
    public void populateData (MahasiswaTr15 dataMhs[], int idxLast) {
        this.dataMahasiswaTr15s = dataMhs;
        this.idxLast = idxLast;
    }
    public void traverseInOrder (int idxStart) {
        if (idxStart <= idxLast) {
            if (dataMahasiswaTr15s[idxStart] != null) {
                traverseInOrder(2*idxStart+1);
                dataMahasiswaTr15s[idxStart].tampilInformasi();
                traverseInOrder(2*idxStart+2);
            }
        }
    }
}
```

➤ Kode Program BinaryTreeArrayMain15

```
public class BinaryTreeArrayMain15 {
    public static void main(String[] args) {
        BinaryTreeArray15 bta = new BinaryTreeArray15();
        MahasiswaTr15 mhs1 = new MahasiswaTr15("2441601212", "Ali", "A", 3.57);
        MahasiswaTr15 mhs2 = new MahasiswaTr15("2441601185", "Candra", "C", 3.41);
        MahasiswaTr15 mhs3 = new MahasiswaTr15("2441601221", "Badar", "B", 3.75);
        MahasiswaTr15 mhs4 = new MahasiswaTr15("2441601220", "Dewi", "B", 3.35);

        MahasiswaTr15 mhs5 = new MahasiswaTr15("2441601131", "Devi", "A", 3.48);
        MahasiswaTr15 mhs6 = new MahasiswaTr15("2441601205", "Ehsan", "D", 3.61);
        MahasiswaTr15 mhs7 = new MahasiswaTr15("2441601170", "Fizi", "B", 3.86);

        MahasiswaTr15[] dataMahasiswas = {mhs1, mhs2, mhs3, mhs4, mhs5, mhs6, mhs7,
        null, null, null};
        int idxLast = 6;
        bta.populateData(dataMahasiswas, idxLast);
        System.out.println("\nInorder Traversal Mahasiswa: ");
        bta.traverseInOrder(0);
    }
}
```

➤ Hasil Program

Inorder Traversal Mahasiswa: NIM: 2441601220 Nama: Dewi Kelas: B IPK: 3.35 NIM: 2441601185 Nama: Candra Kelas: C IPK: 3.41 NIM: 2441601131 Nama: Devi Kelas: A IPK: 3.48 NIM: 2441601212 Nama: Ali Kelas: A IPK: 3.57 NIM: 2441601205 Nama: Ehsan Kelas: D IPK: 3.61 NIM: 2441601221 Nama: Badar Kelas: B IPK: 3.75 NIM: 2441601170 Nama: Fizi Kelas: B IPK: 3.86
--

Pertanyaan Percobaan

1. Apakah kegunaan dari atribut data dan idxLast yang ada di class BinaryTreeArray?
2. Apakah kegunaan dari method populateData()?
3. Apakah kegunaan dari method traverseInOrder()?
4. Jika suatu node binary tree disimpan dalam array indeks 2, maka di indeks berapakah posisi left child dan right child masing-masing?
5. Apa kegunaan statement int idxLast = 6 pada praktikum 2 percobaan nomor 4?
6. Mengapa indeks 2*idxStart+1 dan 2*idxStart+2 digunakan dalam pemanggilan rekursif, dan apa kaitannya dengan struktur pohon biner yang disusun dalam array?

Jawaban

1. Atribut data pada class BinaryTreeArray digunakan untuk menyimpan node-node binary tree dalam bentuk array. Setiap elemen array merepresentasikan satu node pada tree. idxLast digunakan untuk menyimpan indeks terakhir yang berisi data, sehingga program mengetahui batas akhir data yang diproses pada tree.
2. Method populateData() digunakan untuk mengisi array binary tree dengan data mahasiswa yang telah dibuat dari method main(). Selain itu, method ini juga digunakan untuk mengatur nilai idxLast agar program mengetahui jumlah data yang tersimpan dalam array.
3. Method traverseInOrder() digunakan untuk menampilkan data binary tree dengan urutan inorder, yaitu mengunjungi subtree kiri, kemudian root, lalu subtree kanan. Traversal ini biasanya menghasilkan data yang terurut, yaitu kiri → root → kanan.
4. Jika suatu node berada pada indeks 2, maka:
 - left child berada di indeks $2*2+1 = 5$
 - right child berada di indeks $2*2+2 = 6$
5. Statement `int idxLast = 6` digunakan untuk menunjukkan bahwa indeks terakhir yang berisi data adalah indeks ke-6, sehingga data yang tersimpan berjumlah 7 elemen.
6. Indeks $2*idxStart+1$ dan $2*idxStart+2$ digunakan karena pada representasi binary tree dalam array:
 - anak kiri node ke-i berada di indeks $2i+1$
 - anak kanan node ke-i berada di indeks $2i+2$

TUGAS PRAKTIKUM

➤ Kode Program BinaryTree15 new (modifikasi)

```
public void addRekursif(MahasiswaTr15 mahasiswa) {
    root = addRekursif(root, mahasiswa);
}
public NodeT15 addRekursif(NodeT15 current, MahasiswaTr15 mahasiswa) {
    if (current == null) {
        return new NodeT15(mahasiswa);
    }
    if (mahasiswa.ipk < current.mahasiswa.ipk) {
        current.left = addRekursif(current.left, mahasiswa);
    } else if (mahasiswa.ipk > current.mahasiswa.ipk) {
        current.right = addRekursif(current.right, mahasiswa);
    }
    return current;
}
```

```

    }
    public void cariMinIPK() {
        if (isEmpty()) {
            System.out.println("Tree kosong");
            return;
        }
        NodeT15 current = root;
        while (current.left != null) {
            current = current.left;
        }
        current.mahasiswa.tampilInformasi();
    }
    public void cariMaxIPK() {
        if (isEmpty()) {
            System.out.println("Tree kosong");
            return;
        }
        NodeT15 current = root;
        while (current.right != null) {
            current = current.right;
        }
        current.mahasiswa.tampilInformasi();
    }

    public void tampilMahasiswaIPKdiAtas(double ipkBatas) {
        tampilMahasiswaIPKdiAtas(root, ipkBatas);
    }
    public void tampilMahasiswaIPKdiAtas(NodeT15 node, double ipkBatas) {
        if (node != null) {
            tampilMahasiswaIPKdiAtas(node.left, ipkBatas);
            if (node.mahasiswa.ipk > ipkBatas) {
                node.mahasiswa.tampilInformasi();
            }
            tampilMahasiswaIPKdiAtas(node.right, ipkBatas);
        }
    }
}

```

➤ **Kode Program BinaryTreeMain15 new (modifikasi)**

```

System.out.println("\nMahasiswa dengan IPK terkecil:");
bst.cariMinIPK();
System.out.println("\nMahasiswa dengan IPK terbesar:");
bst.cariMaxIPK();
System.out.println("\nMahasiswa dengan IPK di atas 3.50:");
bst.tampilMahasiswaIPKdiAtas(3.50);

```

➤ Hasil Running

Daftar semua mahasiswa (in order traversal):

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54

NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

Pencarian data mahasiswa:

Cari mahasiswa dengan ipk: 3.54: Ditemukan

Cari mahasiswa dengan ipk: 3.22: Tidak ditemukan

Daftar semua mahasiswa setelah penambahan 3 mahasiswa:

InOrder Traversal:

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21

NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37

NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54

NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

PreOrder Traversal:

NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54

NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37

NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

PostOrder Traversal:

NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46

NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57

Penghapusan data mahasiswa

Jika 2 anak, current =

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal)

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21

NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37

NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

Mahasiswa dengan IPK terkecil:

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21

Mahasiswa dengan IPK terbesar:

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

Mahasiswa dengan IPK di atas 3.50:

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

➤ **Kode Program BinaryTreeArray15 new (modifikasi)**

```
public void add(MahasiswaTr15 data) {
    if (idxLast < dataMahasiswaTr15s.length - 1) {
        idxLast++;
        dataMahasiswaTr15s[idxLast] = data;
    } else {
        System.out.println("Array penuh");
    }
}

public void traversePreOrder(int idxStart) {
    if (idxStart <= idxLast) {
        if (dataMahasiswaTr15s[idxStart] != null) {
            dataMahasiswaTr15s[idxStart].tampilInformasi();
            traversePreOrder(2 * idxStart + 1);
            traversePreOrder(2 * idxStart + 2);
        }
    }
}
```

➤ **Kode Program BinaryTreeArrayMain15 new (modifikasi)**

```
System.out.println("\nPreOrder Traversal:");
bta.traversePreOrder(0);
```

➤ **Hasil Running**

Inorder Traversal Mahasiswa:

NIM: 2441601220 | Nama: Dewi | Kelas: B | IPK: 3.35
NIM: 2441601185 | Nama: Candra | Kelas: C | IPK: 3.41
NIM: 2441601131 | Nama: Devi | Kelas: A | IPK: 3.48
NIM: 2441601212 | Nama: Ali | Kelas: A | IPK: 3.57
NIM: 2441601205 | Nama: Ehsan | Kelas: D | IPK: 3.61
NIM: 2441601221 | Nama: Badar | Kelas: B | IPK: 3.75
NIM: 2441601170 | Nama: Fizi | Kelas: B | IPK: 3.86

PreOrder Traversal:

NIM: 2441601212 | Nama: Ali | Kelas: A | IPK: 3.57
NIM: 2441601185 | Nama: Candra | Kelas: C | IPK: 3.41
NIM: 2441601220 | Nama: Dewi | Kelas: B | IPK: 3.35
NIM: 2441601131 | Nama: Devi | Kelas: A | IPK: 3.48
NIM: 2441601221 | Nama: Badar | Kelas: B | IPK: 3.75
NIM: 2441601205 | Nama: Ehsan | Kelas: D | IPK: 3.61
NIM: 2441601170 | Nama: Fizi | Kelas: B | IPK: 3.86